

CDP优化方案 V3.0-05

# OneID 改造 与全链路优化

水平扩展 · 多对象筛选 · 自然语言 Segment · 智能数据底座

2026 年 6 月

信息来源: CDP 专项优化 by Bob + CDP Agent by 国荣

# 1. 方案总览

## 核心目标

- ▶ OneID 实时化：Java 同步服务 → Flink + Redis，识别延迟 P50 < 5ms
- ▶ 多租户水平扩展：Kafka / Flink / Redis / Doris 按租户分级伸缩
- ▶ 多对象筛选：User 单实体 → 人-货-场-线索跨对象圈选（长期）
- ▶ NL Segment：自然语言 → 候选规则 → 引擎校验 → 人工确认

## 技术组件

|                                      |                                       |                                  |                              |
|--------------------------------------|---------------------------------------|----------------------------------|------------------------------|
| <b>Kafka</b><br>多租户事件总线              | <b>Flink</b><br>实时 OneID / 画像 / 宽表    | <b>Redis</b><br>OneID 热层缓存       | <b>Doris</b><br>OLAP 画像 / 圈选 |
| <b>StreamPark</b><br>Flink 实时 Job 管控 | <b>DolphinScheduler</b><br>离线 / 批任务调度 | <b>SQL Engine</b><br>模板化 OLAP 查询 | <b>CDP Agent</b><br>NL 圈人编排层 |

StreamPark 负责实时 Flink Job 启停与扩缩；DolphinScheduler 负责离线导入、批量标签计算、数据同步、Doris 运维及 Job 依赖编排。

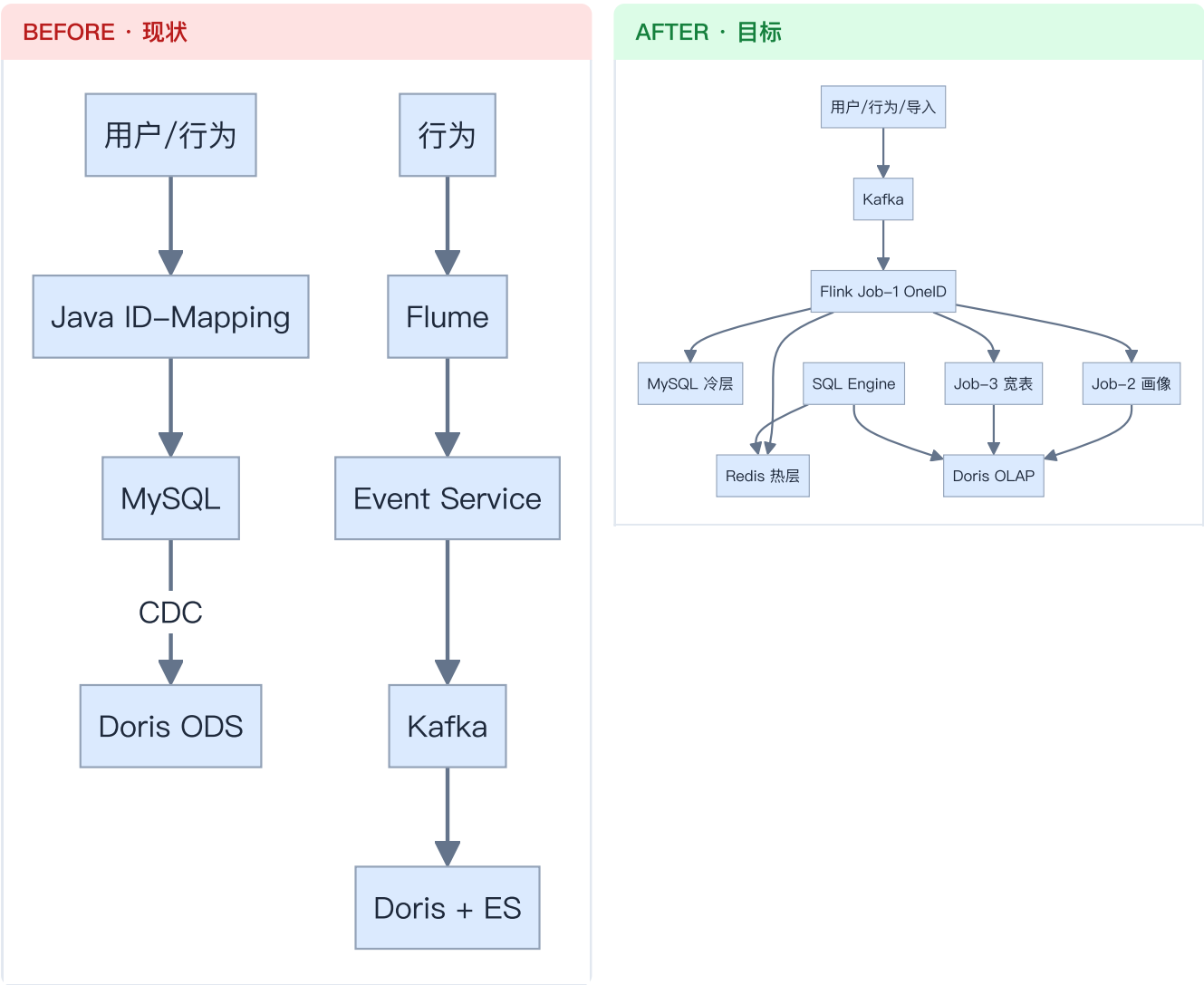
## 目标架构（自上而下）

图 1 · 数据链路层级（接入 → Kafka → Flink → MySQL → Doris）



## 2. 短期目标：OneID 改造与水平扩展

### 2.1 链路 Before / After



| 维度       | Before                     | After                  |
|----------|----------------------------|------------------------|
| OneID 识别 | Java 同步, 10–100ms          | Flink + Redis, < 5ms   |
| 数据写入     | MySQL → CDC → Doris        | Kafka → Flink 直写 Doris |
| 行为链路     | Flume + Event Service + ES | 统一 UserEvent, 无 ES     |
| 横向扩展     | IDM Pod + nginx 路由         | 分区 + 并行度 + 消费组         |
| 任务调度     | 分散 Cron / 手工               | DolphinScheduler 统一编排  |

### 2.2 规模增长与 K8s 节点规划

所有组件部署于 K8s 集群，按数据规模横向扩容。下表为各组件 Pod/节点数参考：

| 规模          | K8s 节点<br>服务器 | Kafka<br>Broker | Redis<br>实例 | Flink<br>TM 节点 | Doris<br>FE + BE | DolphinScheduler<br>Master + Worker | Topic 分区 |
|-------------|---------------|-----------------|-------------|----------------|------------------|-------------------------------------|----------|
| dev (<1万)   | 3             | 1               | 1           | 1              | 1 + 1            | 1 + 1                               | 4        |
| medium (千万) | 6             | 2               | 2           | 2              | 1 + 4            | 1 + 2                               | 8        |
| large (1亿)  | 12            | 3               | 3           | 4              | 2 + 8            | 1 + 3                               | 16       |
| xlarge (2亿) | 20            | 5               | 6           | 8              | 3 + 12           | 2 + 5                               | 32       |

图 2 · K8s 集群组件分布（以 large 规模为例，12 节点）



2.4 数据量与查询性能目标（SLO）

在上述节点规划下，各规模的数据承载与圈选查询性能目标如下。重点关注复杂圈选（20 组筛选条件 + 跨对象 JOIN）的响应时间：

| 规模          | OneID 总量 | 日增事件  | OneID 点查<br>Redis P99 | 单条件圈选<br>Doris | 20 组条件圈选<br>跨对象 JOIN | 圈选并发<br>QPS |
|-------------|----------|-------|-----------------------|----------------|----------------------|-------------|
| dev (<1万)   | <1 万     | <10 万 | <1ms                  | <0.3s          | <1.5s                | 5           |
| medium (千万) | 千万级      | 千万级   | <3ms                  | <1s            | 3—5s                 | 10          |
| large (1亿)  | 1 亿      | 亿级    | <5ms                  | 1—2s           | 6—10s                | 20          |
| xlarge (2亿) | 2 亿      | 数亿    | <5ms                  | 2—3s           | 10—15s               | 30          |

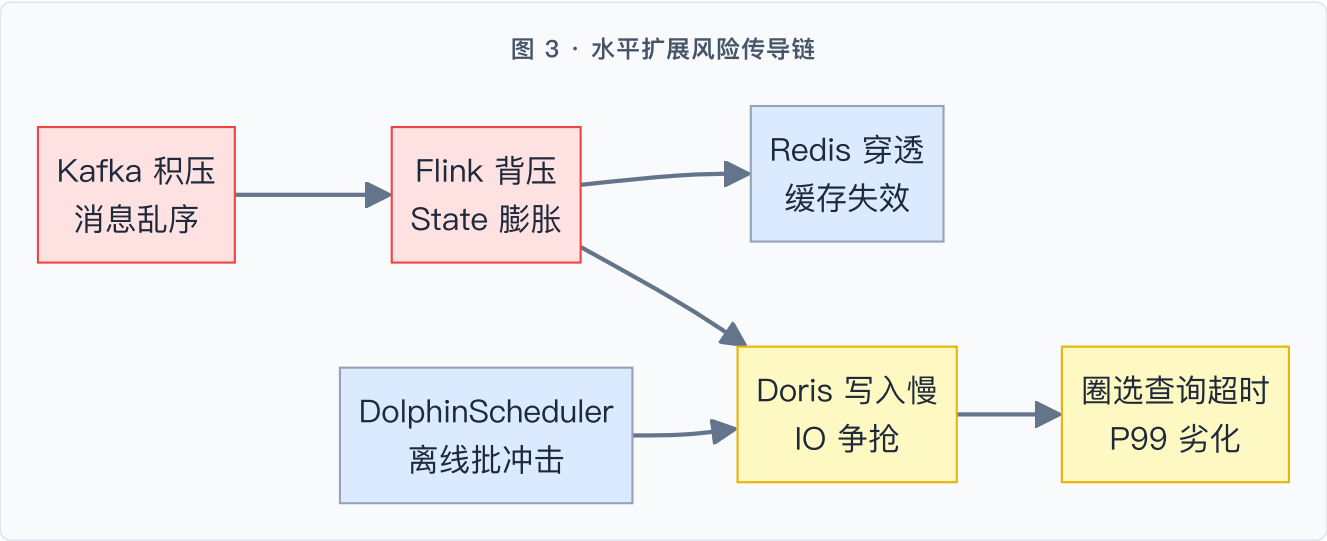
20 组条件圈选 = 用户属性 + 行为 + 标签 + 跨对象关联混合条件；通过 tenant\_id 分区裁剪、分桶 ≥16、SQL Engine 模板化与超时熔断保障 P99；超 large 规模建议预计算高频 Segment 物化结果以将复杂圈选降至秒级。

2.3 水平扩展带来的问题与对策

扩容不是简单加节点，以下四类问题是主要风险点：

| 组件               | 典型问题                                                                                                       | 风险 | 对策                                                                                                             |
|------------------|------------------------------------------------------------------------------------------------------------|----|----------------------------------------------------------------------------------------------------------------|
| Redis            | ① 大租户热 Key 打满单分片<br>② merge 后多副本缓存不一致<br>③ 扩容后冷数据穿透 Doris<br>④ LRU 淘汰导致命中率骤降                               | 高  | 大租户独立 Redis Cluster；merge 后主动失效 channel key；<br>写时双写 + 短 TTL 兜底；监控命中率 < 90% 告警                                 |
| Kafka            | ① 共享 Topic 仅保证同 tenant 有序<br>② create/merge 事件跨分区乱序<br>③ 扩分区后 rebalance 重复消费<br>④ link_keys 跨分区关联竞态        | 高  | merge 相关事件 key 统一为 one_id；<br>幂等写入 + merge_log 去重；<br>rebalance 期间开启幂等检查窗口                                     |
| Flink            | ① Doris 写入慢 → 背压 → Kafka 积压<br>② keyBy 热点 subtask 倾斜<br>③ RocksDB State 过大 checkpoint 超时<br>④ 三 Job 链路延迟叠加 | 高  | Async Sink + 批量写入 Doris；<br>热点 key 加盐再聚合；<br>State TTL + 增量 checkpoint；<br>各 Job 独立监控 lag，超 60s 告警             |
| Doris            | ① 小租户共享库全表扫描<br>② 分桶倾斜导致长尾查询<br>③ 实时写入与圈选查询抢 IO<br>④ 宽表 JOIN 内存溢出                                          | 中  | 强制 tenant_id 分区裁剪；<br>按数据量重估 BUCKETS (≥16)；<br>读写分离：写入走 Routine Load，查询走 Follower；<br>圈选走 SQL Engine 模板 + 超时熔断 |
| DolphinScheduler | ① 离线批与实时流资源争抢<br>② 多租户批任务并发打满 Doris<br>③ 任务依赖失败级联                                                          | 中  | 离线任务限流 + 租户级配额；<br>批流分时调度（凌晨批 / 白天实时）；<br>关键路径设置重试 + 告警 + 人工兜底                                                 |

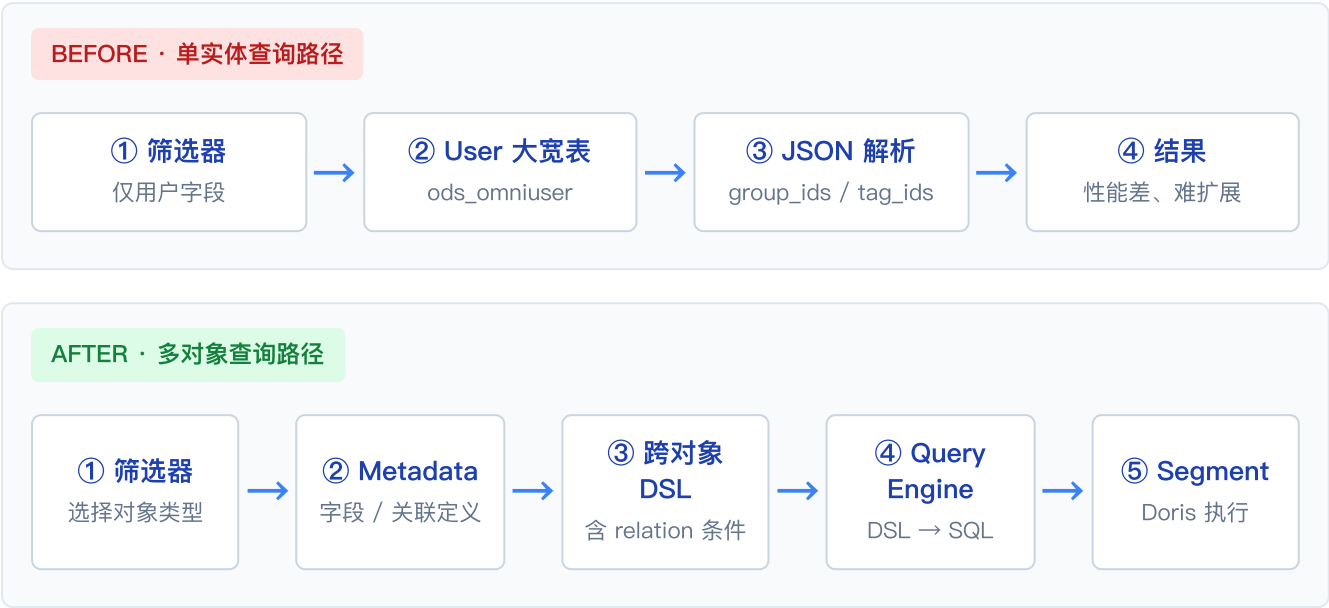
图 3 · 水平扩展风险传导链



3. 长期目标：多对象筛选

| 维度   | Before                         | After                                   |
|------|--------------------------------|-----------------------------------------|
| 核心实体 | 仅 User (OneID)                 | User + Lead + Account + Product + Store |
| 存储   | 单表大 JSON (group_ids / tag_ids) | 对象独立表 + object_relations                |
| 筛选   | 用户属性 + 行为                      | 跨对象 JOIN + 关联行为                         |
| 竞品差异 | 与火山 VeCDP 持平                   | 多对象圈选 (火山不支持)                           |

3.1 查询路径对比 (图 4)



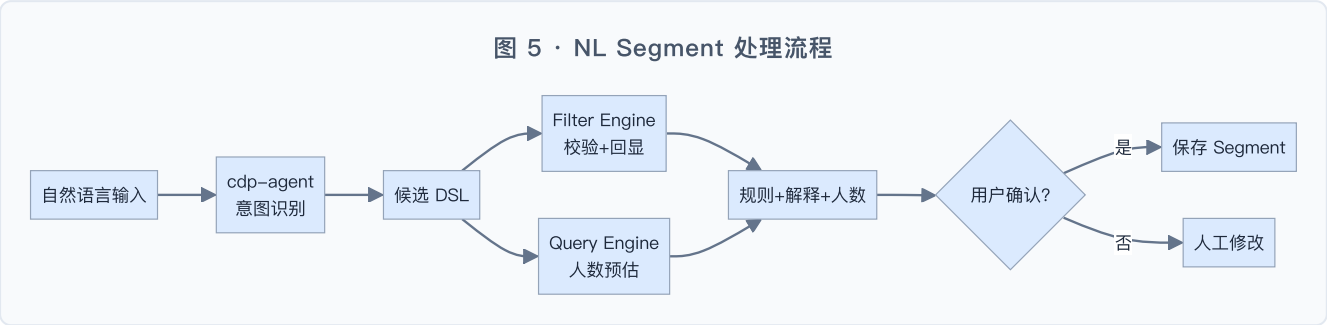
|         | User         | Lead         | Account | Product     | Store     |
|---------|--------------|--------------|---------|-------------|-----------|
| User    | —            | ← belongs_to | owns →  |             | visited → |
| Lead    | belongs_to → | —            |         |             |           |
| Account |              |              | —       | purchased → |           |
| Product |              |              |         | —           |           |
| Store   | ← visited    |              |         |             | —         |

矩阵中蓝色单元格表示已定义的关联关系，存储于 `object_relations` 表，`Query Engine` 通过 `JOIN` 实现跨对象筛选。

**跨对象圈选示例：**「地址在上海、公司规模 > 500 的线索，且关联用户带有 VIP 标签」  
→ DSL `object=lead + relation.belongs_to(user.tags contains vip)` → Doris JOIN 执行

#### 4. 自然语言 Segment (CDP Agent)

- ▶ LLM 只负责意图理解，不直接拼 SQL、不直接发规则
- ▶ 候选 DSL 必须经过 Filter / Query Engine 校验 + 人数预估
- ▶ 用户确认后才走现有规则保存链路



| 阶段      | 能力                              | 周期     |
|---------|---------------------------------|--------|
| Phase 1 | NL 圈人 + 人数预估 + 多轮澄清 + 只读 MCP    | 约 2 个月 |
| Phase 2 | 跨对象 NL 圈人 + 规则体检 + 匹配诊断         | 约 2 个月 |
| Phase 3 | Doris Hybrid Search + 选项/文本语义召回 | 约 2 个月 |



## 5. 实施时间线（10 月前功能补足）

6—9 月按功能线与性能线双轨并行推进，10 月前完成 CDP 核心功能补足与底层性能改造：

|     | 6 月            | 7 月              | 8 月                           | 9 月                  |
|-----|----------------|------------------|-------------------------------|----------------------|
| 功能线 | 超级租户<br>数据权限隔离 | ABM<br>客户档案      | 自定义对象<br>对象 / 字段自定义           | —                    |
| 性能线 | —              | 现状改造<br>针对现有架构优化 | 重构 OneID<br>Flink + Redis 实时化 | 全链路优化<br>性能调优 + 水平扩展 |

| 轨道  | 月份  | 交付内容                                                    |
|-----|-----|---------------------------------------------------------|
| 功能线 | 6 月 | 超级租户：数据权限隔离，租户间数据资源与可见范围彻底隔离，支撑多租户安全合规                  |
|     | 7 月 | ABM 客户档案：建立 Account 维度档案体系，支撑面向客户的精细化运营                 |
|     | 8 月 | 自定义对象：开放对象与字段自定义能力，适配不同行业的数据模型                          |
| 性能线 | 7 月 | 针对现有架构做改造，解决当前性能瓶颈与稳定性问题                                |
|     | 8 月 | 重构 OneID：Java 同步服务 → Flink + Redis 实时化，识别延迟降至 < 5ms     |
|     | 9 月 | 全链路性能优化：链路调优 + 水平扩展（Kafka / Flink / Redis / Doris 分级伸缩） |

功能线优先补齐对外能力（超级租户 → ABM → 自定义对象）；性能线在 7 月起承接现状改造，并以 8 月 OneID 重构为关键里程碑，为 9 月全链路水平扩展奠定基础（详见第 2 章）。

## 6. 智能数据底座（10—12 月 · 面向 Agent）

在 CDP（OneID + 多对象 + 圈选）之上构建 **智能数据底座**：将数据接入、加工、语义化能力沉淀为标准服务，统一封装为 MCP / API，为上层 Agent 提供可直接调用的数据底座。10—12 月分三步交付。

### 6.1 面向 Agent 的基础服务



### 6.2 实施时间线（10—12 月）

|      | 10 月                     | 11 月                   | 12 月                          |
|------|--------------------------|------------------------|-------------------------------|
| 数据底座 | 可视化 ETL → 多对象<br>拖拽式接入加工 | 多模态非结构化处理<br>文档/图像/音视频 | NL 语义层优化<br>Segment/Tag/Group |

| 月份                                                                     | 基础服务                        | 交付内容                                                                   |
|------------------------------------------------------------------------|-----------------------------|------------------------------------------------------------------------|
| 10 月                                                                   | 可视化 ETL → 多对象               | 拖拽式数据接入与转换，直接写入多对象模型（User / Lead / Account / Product / Store），降低数据接入门槛 |
| 11 月                                                                   | 多模态非结构化数据处理                 | 文档 / 图像 / 音视频等非结构化数据解析 → 结构化字段抽取 + 向量化入库，扩充画像与圈选维度                     |
| 12 月                                                                   | NL Segment / Tag / Group 优化 | 自然语言生成与优化 Segment、Tag、Group，提升语义召回与圈人准确率                               |
| 贯穿全程 · Agent 数据底座服务：上述能力统一封装为 MCP / API，对外提供标准化数据供给，支撑上层 Agent 直接编排调用。 |                             |                                                                        |

智能数据底座承接 6—9 月 CDP 功能与性能改造成果，将「数据接入 → 加工 → 语义化 → 服务化」全链路能力对 Agent 开放，是 CDP 向 AI 原生数据平台演进的关键一步。